
ExoHabitAI

NASA Exoplanet Habitability Predictor

MLESTONE 3 REPORT

Backend & Frontend — Theoretical Documentation

Powered by Tuned Random Forest ML • 98.89% Accuracy • 22 Features

1. Project Overview

1.1 Introduction

ExoHabitAI is an advanced machine learning system developed to predict the habitability of exoplanets using real astronomical data from NASA's Exoplanet Archive. The system combines a trained Tuned Random Forest model with a Flask REST API backend and an interactive web-based frontend, forming a complete end-to-end pipeline for exoplanet habitability analysis.

The project addresses the scientific challenge of identifying potentially habitable worlds beyond our solar system. Using 22 key astronomical features including planetary radius, mass, orbital period, equilibrium temperature, and stellar parameters, the model achieves 98.89% accuracy with 97.5% recall.

1.2 Project Objectives

- Predict whether an exoplanet is habitable or non-habitable using supervised machine learning.
- Build a robust Flask REST API backend capable of processing single and batch prediction requests.
- Design an immersive, interactive frontend web application for scientists and researchers.
- Provide a ranked leaderboard of the most habitable exoplanet candidates discovered so far.
- Deliver a live analytics dashboard tracking all predictions and model performance metrics.

1.3 Technology Stack

Component	Technology Used
ML Framework	Scikit-learn (Random Forest Classifier)
Backend API	Flask (Python) with Flask-CORS
Data Processing	Pandas, NumPy, Joblib
Frontend	HTML5, CSS3, JavaScript (ES6+)
Charting Library	Chart.js 4.4.1
Data Source	NASA Exoplanet Archive
Model Persistence	Joblib (.pkl serialization)
Fonts & Icons	Google Fonts (Orbitron, Exo 2), Font Awesome 6

2. Backend — Theoretical Documentation

The backend of ExoHabitAI is built using Python and the Flask micro-framework. It serves as the machine learning inference engine, exposing REST API endpoints that the frontend communicates with. This section provides a complete theoretical explanation of all backend components.

2.1 Backend Architecture

The backend follows a modular, layered architecture consisting of three logical components:

1. Utility Layer (utils.py) — Defines required features, loads the ML model, and validates incoming input data.
2. Application Layer (app.py) — Flask application that defines HTTP routes, processes requests, and returns JSON responses.
3. Model Layer (/models/random_forest.pkl) — The pre-trained Tuned Random Forest model saved as a serialized Joblib file.

This separation of concerns ensures that prediction logic, validation logic, and routing logic remain independent, making the codebase maintainable and testable.

2.2 utils.py — Utility Module

The utils.py module acts as the backbone of the backend. It is responsible for three critical operations: defining what input features the model expects, loading the trained model from disk, and validating any incoming JSON data before a prediction is made.

2.2.1 Feature Definition — FEATURES List

The FEATURES constant is a Python list containing all 22 feature names that the ML model requires as input. The list is used as a reference for input validation and as column headers when constructing the Pandas DataFrame passed to the model.

Category	Features
Planetary	pl_rade, pl_bmasse, pl_orbper, pl_orbsmax, pl_eqt
Stellar	st_teff, st_rad, st_mass, st_met, st_lum
Spectral Type (Binary)	spec_F, spec_G, spec_K, spec_M, spec_m
Derived Scores	temp_score, dist_score, lum_score, stellar_temp_score, stellar_mass_score, stellar_compatibility_index, orbital_stability

2.2.2 Model Loading — `load_model()`

The `load_model()` function uses Python's `pathlib` library to construct a platform-independent file path to the serialized model at `models/random_forest.pkl`. It then uses `joblib.load()` to deserialize the model object into memory. If the model file is missing or corrupt, the function catches the exception, logs an error message, and returns `None`. The model is loaded once at application startup to ensure fast prediction response times.

2.2.3 Input Validation — `validate_input()`

1. The `validate_input()` function performs a three-stage validation process on incoming JSON data before the model makes a prediction:
2. Stage 1 — Null Check: Verifies the incoming data is not `None`. Returns an error if no JSON body was received.
3. Stage 2 — Completeness Check: Iterates through all 22 feature names and verifies each is present in the data dictionary. Returns an error specifying which feature is missing.
4. Stage 3A — Numeric Validation: For each of the 17 numeric features (`pl_rade`, `pl_eqt`, `st_teff`, etc.), checks the value is not null and can be converted to a Python float.
5. Stage 3B — Binary Validation: For each of the 5 spectral type features (`spec_F`, `spec_G`, `spec_K`, `spec_M`, `spec_m`), validates the value is 0, 1, '0', '1', `True`, or `False`.

2.3 `app.py` — Flask Application

The `app.py` file is the main entry point of the backend. It creates the Flask application, configures CORS, loads the ML model and ranked dataset at startup, and defines three HTTP endpoint routes.

2.3.1 Application Initialization

1. Creates the Flask app instance and enables CORS using Flask-CORS to allow the frontend (running on a different port) to make API calls without browser security restrictions.
2. Sets up Python logging at INFO level.
3. Calls `load_model()` from `utils.py` to load the Random Forest classifier into memory.
4. Loads the pre-ranked exoplanet CSV dataset (`data/habitability_ranked.csv`) into a Pandas DataFrame for use by the `/rank` endpoint.

2.3.2 Home Route — GET `/`

The root endpoint returns a simple JSON health-check response confirming the backend server is running. Used during development to verify the server is accessible.

```
Response: { "status": "success", "message": "ExoHabitAI Backend Running" }
```

2.3.3 Prediction Endpoint — POST /predict

The /predict endpoint is the core of the backend. It receives planetary and stellar parameter data as a JSON body, validates the input, runs the ML model, and returns a structured habitability prediction response. The endpoint processes the request in the following sequence:

1. Parse JSON: Extracts the JSON body using `request.get_json()`.
2. Validate Input: Calls `validate_input()`. If invalid, returns 400 Bad Request immediately.
3. Model Check: Verifies the ML model was loaded. Returns 500 if unavailable.
4. Construct DataFrame: Converts the input dictionary into a single-row Pandas DataFrame with columns ordered exactly as the FEATURES list.
5. Type Conversion: Converts all values to float using `astype(float, errors='ignore')`.
6. Prediction: Calls `model.predict()` to get a binary class prediction (0 = non-habitable, 1 = habitable).
7. Probability Score: Calls `model.predict_proba()` to get the probability for each class. Index [0][1] is the habitability confidence score.
8. Threshold Logic: If habitability probability is below 0.9, the prediction is overridden to 0 (non-habitable).
9. Return Response: Returns a JSON object with `prediction_result` (0 or 1) and `confidence_score`.

2.3.4 Ranking Endpoint — GET /rank

The /rank endpoint returns a sorted list of exoplanet candidates ranked by pre-computed habitability scores. It accepts a limit query parameter (default: 10). The endpoint verifies the ranked DataFrame is not empty, that the Rank column exists, sorts ascending by Rank, selects the top N rows, converts NaN to None for JSON safety, and returns results as a list of dictionaries.

2.4 Machine Learning Model — Tuned Random Forest

The core prediction capability is a Tuned Random Forest Classifier trained on real NASA exoplanet data. Random Forest is an ensemble learning method that builds many decision trees and outputs the mode of their individual predictions. It is well-suited for this task because it handles high-dimensional data with mixed numeric and binary features, is robust to overfitting through bootstrap aggregation, provides built-in feature importance scores, and produces well-calibrated probability scores via `predict_proba()`.

2.4.1 Model Performance Metrics

Metric	Score	Formula	Interpretation
Accuracy	98.89%	Correct / Total	Near-perfect overall classification rate
Recall (Sensitivity)	97.5%	$TP / (TP + FN)$	Finds 97.5% of all truly habitable planets
F1-Score	98.73%	$2 * (P * R) / (P + R)$	Excellent precision-recall balance
ROC-AUC	0.9994	Area under ROC curve	Near-perfect class discrimination
Decision Threshold	0.9	$P(\text{habitable}) \geq 0.9$	High-confidence habitability cutoff

2.4.2 The 0.9 Decision Threshold

A key design decision is the use of a 0.9 decision threshold instead of the default 0.5. A planet is only classified as habitable if the model is at least 90% confident. This high threshold was chosen because false positives — incorrectly classifying a non-habitable planet as habitable — are scientifically costly. The tradeoff is a small reduction in recall, but every planet flagged as habitable by ExoHabitAI is a genuinely strong scientific candidate.

2.5 Error Handling and HTTP Status Codes

HTTP Status	Meaning in ExoHabitAI
200 OK	Prediction or ranking returned successfully
400 Bad Request	Input validation failed — missing or invalid feature value
500 Internal Server Error	ML model not loaded or ranked data unavailable

3. Frontend — Theoretical Documentation

The ExoHabitAI frontend is a single-page web application (SPA) built with HTML5, CSS3, and vanilla JavaScript ES6+. It provides an immersive, space-themed user interface for interacting with the Flask backend, visualizing prediction results, and exploring exoplanet data. The entire frontend is contained in three files: `index.html`, `style.css`, and `script.js`.

3.1 Frontend Architecture

The frontend is structured as a single HTML page with six distinct sections, each representing a major feature area of the application. A sticky navigation bar allows smooth scrolling between sections. Each section communicates with the Flask backend asynchronously using the JavaScript Fetch API.

Section	Purpose
Section 1 — Home (Hero)	Landing page with animated planet, model statistics, and mission cards
Section 2 — About	Project overview, objectives, model architecture, and technology stack
Section 3 — Batch Analysis	CSV upload and JSON paste interface for analyzing multiple planets at once
Section 4 — Ranking	Leaderboard table of top exoplanet candidates with filtering controls
Section 5 — Predict	Single planet prediction form with 22 input fields and result visualization
Section 6 — Analytics Dashboard	Live charts, statistics, and prediction history tracking

3.2 Navigation System

The navigation bar is a fixed-position element that remains visible at the top of the screen as the user scrolls. A JavaScript scroll event listener applies a 'scrolled' CSS class that triggers a backdrop-blur glass-morphism effect once the user scrolls past 50 pixels. The active navigation link is highlighted dynamically based on which section is currently in the viewport using `getBoundingClientRect()`.

3.3 Canvas-Based Visual Engine

A major technical feature is the multi-layered canvas animation system, called the '3D Universe Engine'. Multiple HTML5 canvas elements are overlaid across different sections to create immersive space-themed visual effects without using any external 3D libraries. All canvases are flagged as pointer-events:none so they never interfere with user interactions.

Canvas Effect	Section and Description
Animated Starfield	Global background — 320 pulsing stars with nebula glow blobs
Deep Space Nebula + Wormhole	Hero — layered star parallax with rotating wormhole vortex rings
3D Planet Globe	Hero — shaded sphere with 4 orbiting colored moons and trail effects
Geometric Network Graph	About — animated nodes connected by edges, floating slowly
Data Stream / Matrix Rain	Batch — falling numeric digits with pulsing grid overlay
Warp Speed Starfield	Ranking — radial star motion simulating light-speed travel
Solar System Orrery	Predict — 4 planets orbiting a central star with trail effects
Rotating Wireframe Globe	Analytics — latitude/longitude sphere with glowing equator

3.4 Hero Section

The Hero Section is the first screen a user sees when they open ExoHabitAI. Key elements include: a badge pill showing model metrics (98.89% accuracy, 97.5% recall), an animated headline with gradient text, two call-to-action buttons linking to the Predict and Ranking sections, a scrolling ticker strip with six key model statistics, three feature cards with 3D flip-in scroll animation, and six animated count-up stat counters that animate from zero when scrolled into view.

3.5 Batch Analysis Section

The Batch Analysis section allows users to analyze multiple exoplanets in a single operation, supporting two input methods toggled via a tab interface.

3.5.1 CSV Upload Mode

Users can drag-and-drop or click-to-browse a CSV file containing multiple planet rows. The JavaScript reads the CSV file using the FileReader API, parses it into an array of planet objects, and sends each row as a separate POST request to the /predict Flask endpoint. Results are collected and displayed in a summary table showing habitability, confidence score, and planet name for each row.

3.5.2 JSON Input Mode

Users can paste a raw JSON object into a textarea in the format: `{ 'planets': [{...}, {...}] }`. The JavaScript parses this JSON, iterates through the planets array, and sends each planet as a separate API call to `/predict`. Pre-loaded sample buttons (Habitable and Non-Habitable) populate the input with example test values.

3.6 Exoplanet Ranking Section

The Ranking Section displays a sorted leaderboard of pre-analyzed exoplanet candidates fetched from the Flask `/rank` endpoint. The ranking table displays: Rank position, Planet Name, Radius (Earth radii), Mass (Earth masses), Orbital Period (days), Equilibrium Temperature (K), Stellar Temperature (K), Habitability Score, and Status badge. Filter controls allow users to specify the number of results to display, filter by habitable or non-habitable status, and set a minimum habitability score threshold.

3.7 Single Planet Prediction Section

The Predict Section is the primary interactive feature. It presents a comprehensive input form with 22 fields organized into four logical groups, sends the data to the Flask `/predict` endpoint, and renders a rich result visualization.

3.7.1 Input Form Structure

Group	Fields	Description
Planetary Parameters	pl_rade, pl_bmasse, pl_orbper, pl_orbsmax, pl_eqt	Core planet measurements — radius, mass, orbit, temperature
Stellar Parameters	st_teff, st_rad, st_mass, st_met, st_lum	Host star properties — temperature, size, mass, metallicity, luminosity
Spectral Types	spec_F, spec_G, spec_K, spec_M, spec_m	Binary checkboxes indicating host star spectral classification
Habitability Scores	temp_score, dist_score, lum_score, stellar_temp_score, stellar_mass_score, stellar_compatibility_index, orbital_stability	Computed 0–1 scores derived from raw features during preprocessing

3.7.2 Prediction Result Display

After submitting the form, the prediction result is displayed in a multi-component result panel:

- Result Banner: Colour-coded banner (green for habitable, red for non-habitable) with the prediction label.
 - 3D Planet Preview: Real-time canvas rendering where colour and appearance change based on equilibrium temperature.
-

-
- **Score Boxes:** Habitable and Non-Habitable confidence score cards displayed side-by-side.
 - **Temperature Zone Gauge:** Horizontal bar with five zones (Frozen, Cold, Habitable, Warm, Hot) with a moving needle indicator.
 - **Confidence Level Bar:** Animated progress bar filling from 0% to the model's confidence score, colour-coded from red to green.
 - **Score Breakdown Bars:** Individual animated bars showing each of the seven habitability input scores as visual percentage bars.

3.8 Analytics Dashboard Section

The Analytics Dashboard tracks all predictions made during the current browser session and presents them in an interactive multi-chart dashboard that auto-syncs whenever a new prediction is made.

3.8.1 Statistics Cards

Four glowing stat cards display session totals: Total Analyzed, Habitable Candidates, Model Accuracy (98.89%), and Recall Rate (97.5%). Four model insight cards show Features (22), F1-Score (98.73%), ROC-AUC (0.9994), and Threshold (0.9).

3.8.2 Chart.js Visualizations

- **Feature Importance:** Horizontal bars showing relative importance of each feature in driving predictions.
- **Habitability Score Distribution:** Area chart showing probability score distribution across all analyzed planets.
- **Stellar Type Distribution:** Bar chart breaking down analyzed planets by host star spectral type.
- **Prediction History Trend:** Line chart plotting habitability confidence scores over the last 20 predictions.
- **Mass vs Orbital Distance:** Scatter plot where each point represents a planet positioned by mass and semi-major axis.
- **Classification Results Donut:** Pie-style chart showing ratio of habitable to non-habitable predictions.

3.8.3 Data Export Functions

The dashboard provides three export options: PDF Report (exports an HTML-formatted report with all statistics), CSV Data (downloads prediction history as comma-separated values), and JSON (exports the complete session history with metadata). A Reset button clears all session history and resets all charts to zero.

3.9 Scroll-Triggered Animations

ExoHabitAI uses the IntersectionObserver API to trigger animations when elements enter the browser viewport:

- Count-Up Counters: Elements animate their numeric value from 0 to their target over 1800ms with ease-out cubic easing when scrolled into view.
- 3D Card Flip-In: Feature cards trigger a CSS 3D perspective flip animation on scroll entry, then transition to a gentle floating animation.
- Title Underlines: Section titles trigger an animated expanding underline effect when they enter the viewport.

3.10 Responsive Design

The frontend uses CSS Grid and Flexbox layouts throughout, with CSS custom properties for consistent theming. Media queries adapt the layout for mobile screens, collapsing multi-column grids into single columns and replacing the navigation bar with a hamburger menu button that toggles link visibility.

4. Frontend-Backend API Communication

The ExoHabitAI frontend communicates with the Flask backend using the JavaScript Fetch API with `async/await` syntax. All API calls target `http://localhost:5000`, the default Flask development server address. Flask-CORS on the backend allows these cross-origin requests.

4.1 Single Planet Prediction Flow

1. The form's submit event is intercepted using `addEventListener('submit')` with `preventDefault()` to stop page reload.
2. All 22 input field values are collected from the DOM using `document.getElementById()` calls.
3. Spectral type checkbox values are converted to 0 or 1 using the `checked` property.
4. All values are assembled into a JavaScript object matching the FEATURES list structure.
5. A POST request is sent to `/predict` with `Content-Type: application/json` and the data JSON-stringified in the body.
6. The response JSON is parsed and `prediction_result` and `confidence_score` values are extracted.
7. All result UI components are updated: banner, confidence bar, temperature gauge, planet canvas, and score bars.
8. The prediction is recorded in the session analytics history array and all dashboard charts are refreshed.

4.2 Batch Processing Flow

For batch analysis via CSV, the file is read using `FileReader.readAsText()`, then parsed line by line into planet data objects. Each planet is sent as an individual POST request to `/predict`.

Promises are collected using `Promise.all()` so all predictions run in parallel and results are displayed simultaneously once all responses are received.

4.3 Ranking Data Flow

The ranking table is populated by a GET request to `/rank?limit=N`. The returned JSON array is iterated to create HTML table rows dynamically using string template literals, which are injected into the table body using `innerHTML`. Each row includes a colour-coded status badge based on the `prediction_result` field.

5. Frontend Screenshots

The following section presents step-by-step screenshots of the ExoHabitAI frontend interface, illustrating each major feature and section of the web application.

Screenshot 1 — Hero Section — Home Page

The landing page showing the animated space background, the rotating 3D planet with orbiting moons, the main headline 'Discover the Next Habitable World', the model accuracy badge pill (98.89% accuracy, 97.5% recall), the two call-to-action buttons, and the scrolling ticker strip showing model performance statistics.

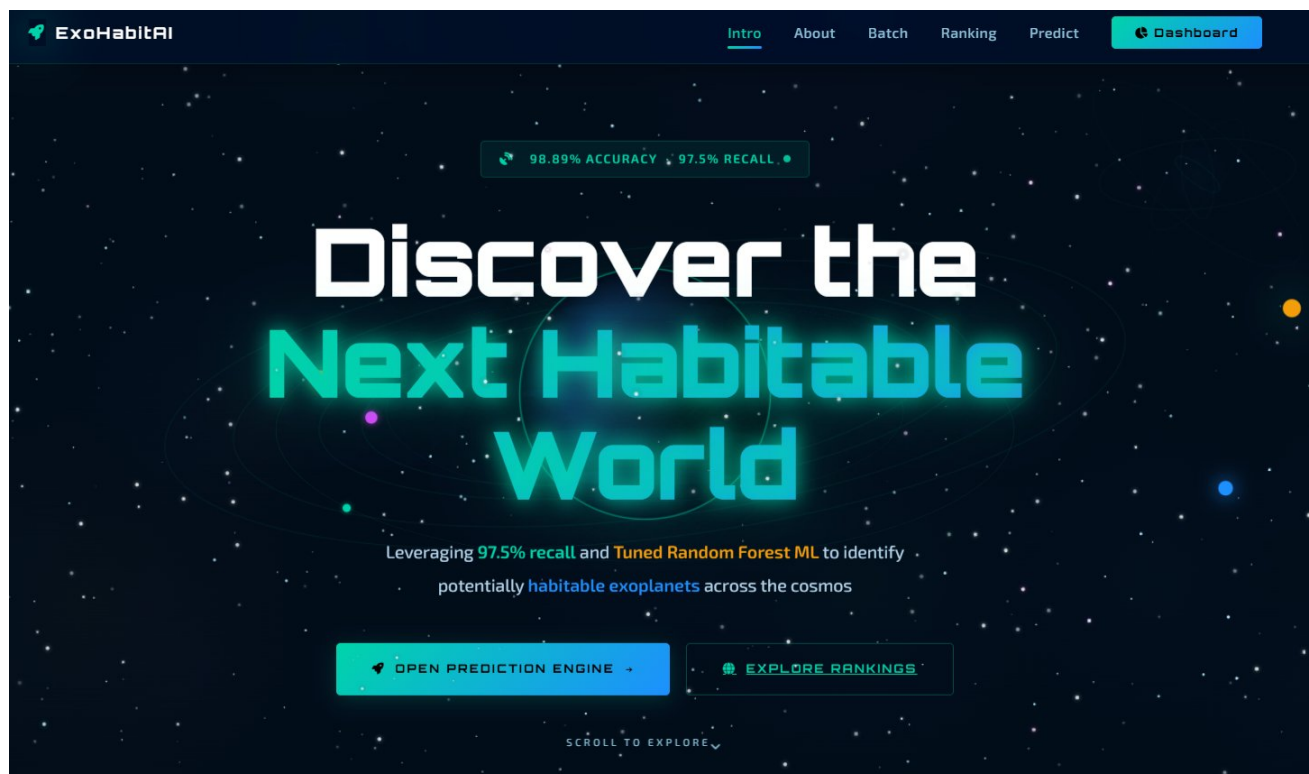


Figure 1: Hero Section — Home Page

Screenshot 2 — Mission Cards and Statistics Band

The three feature cards (AI-Powered Prediction, Key Astronomical Features, Flask REST API) displayed in a three-column grid with their 3D flip-in scroll animation, followed by the six animated count-up statistic cards showing model performance numbers (98.89% accuracy, 97.5% recall, 98.73% F1-score, 22 features, 0.9994 ROC-AUC, 0.9 threshold).

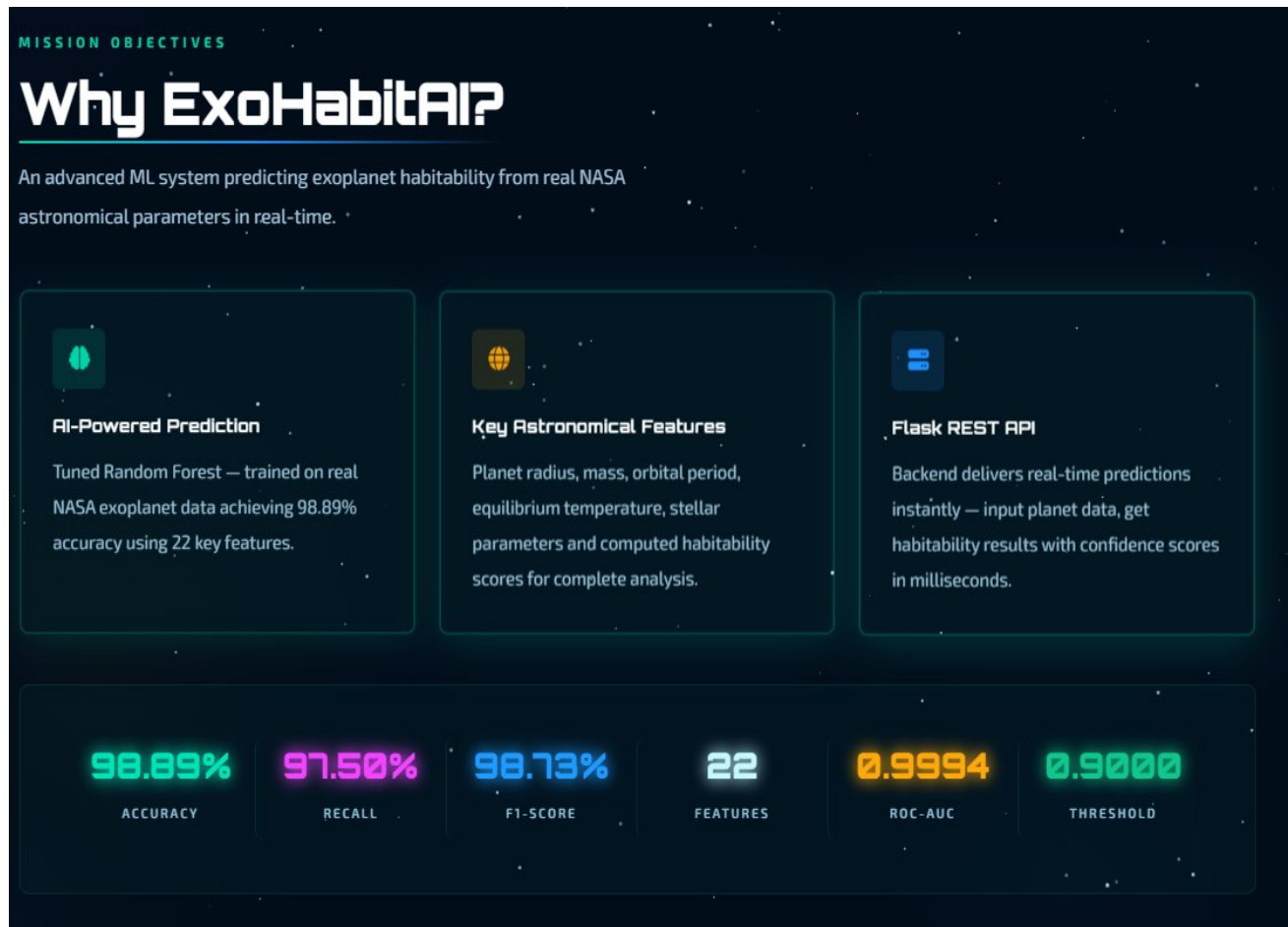


Figure 2: Mission Cards and Statistics Band

Screenshot 3 — About Section — Project Overview

The About section showing the project description and the "About ExoHabitAI" header with the project overview badge, followed by the 'What is ExoHabitAI?' description explaining the ML system's purpose.

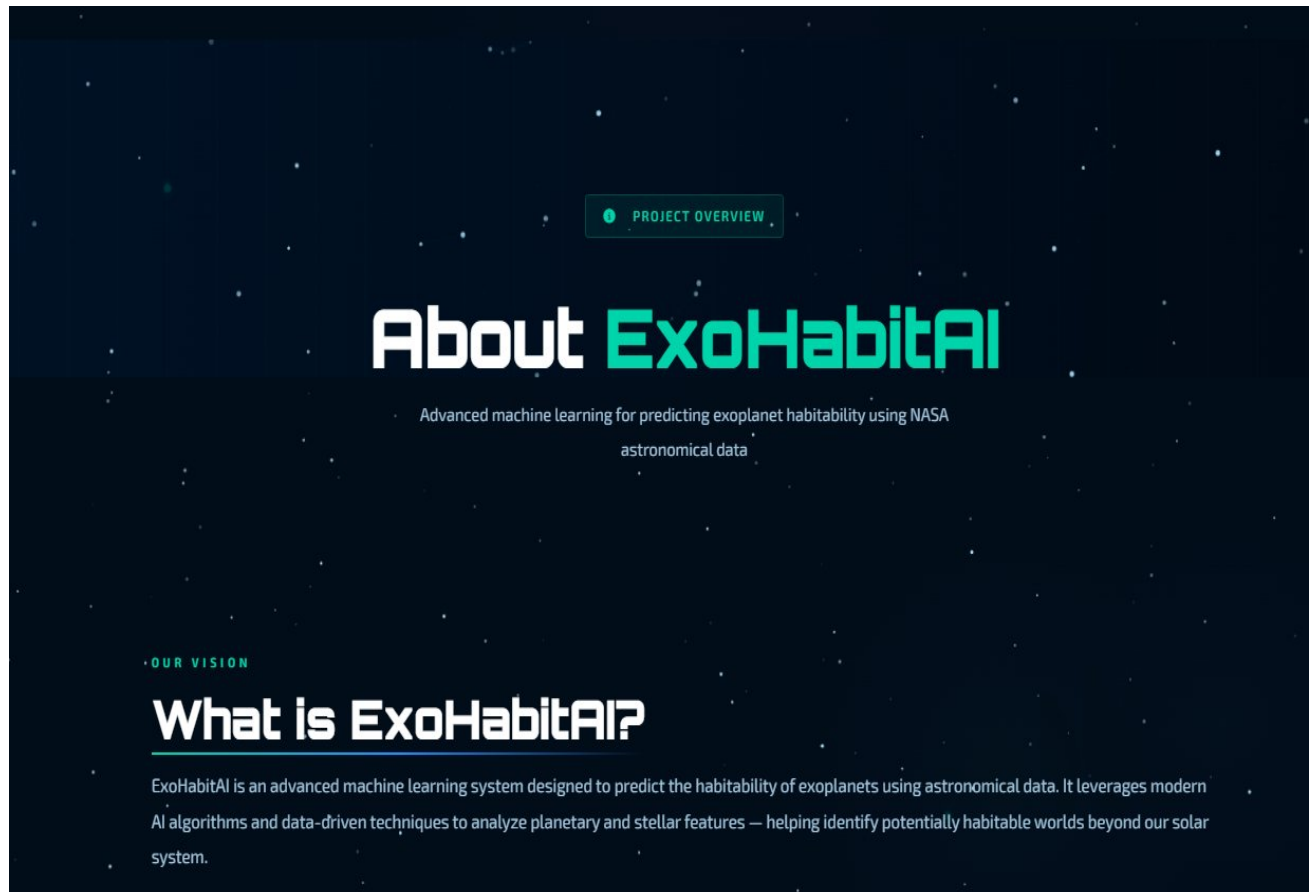


Figure 3: About Section — Project Overview

Screenshot 4 — About Section — Objectives

The objectives grid from the About section, showing the four project goal cards: Predict Habitability, Analyze Space Data, Real-Time Predictions, and Scientific Exploration, each with descriptive text.

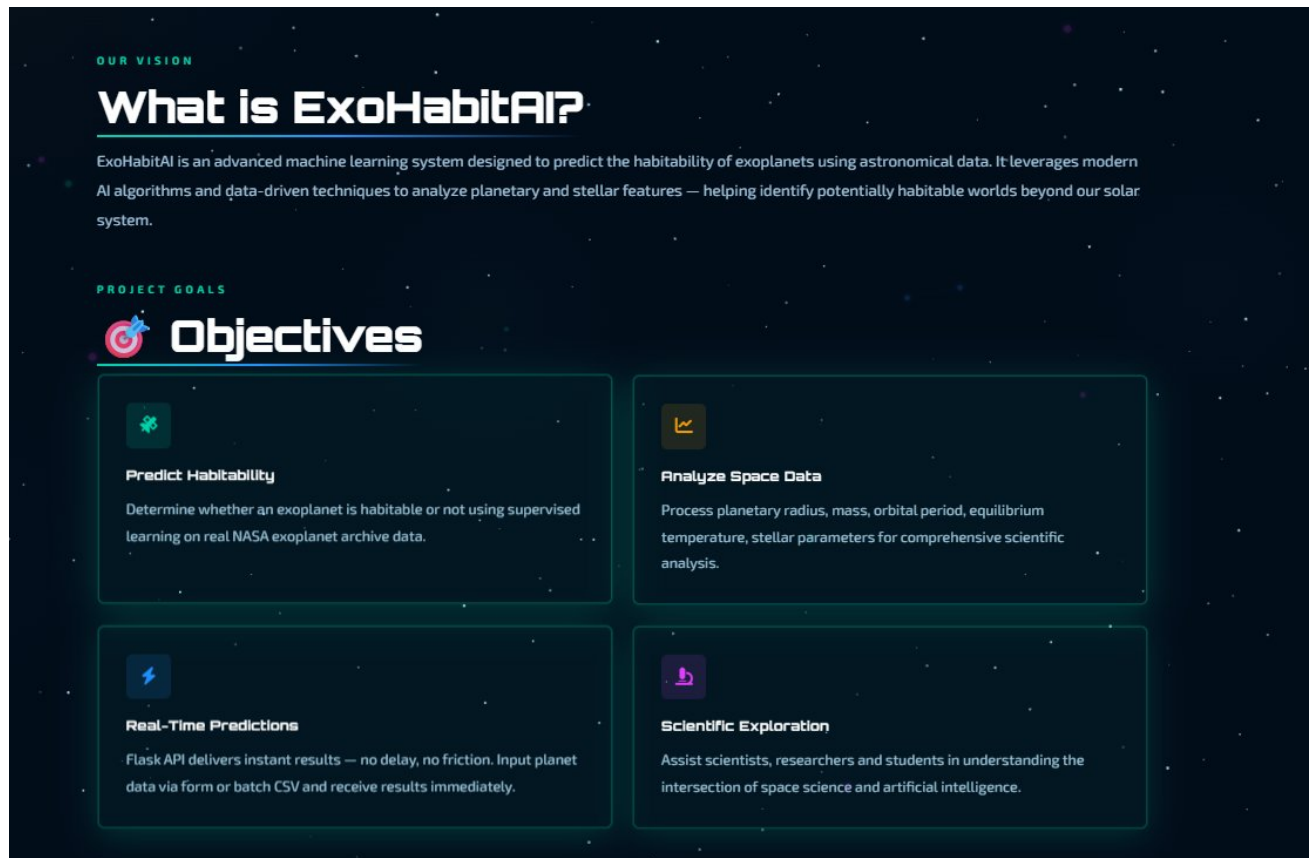


Figure 4: About Section — What is ExoHabitAI & Objectives

Screenshot 5 — Objectives Cards

Full view of the four project objectives cards showing the goal icons and descriptions for each core feature of the system.

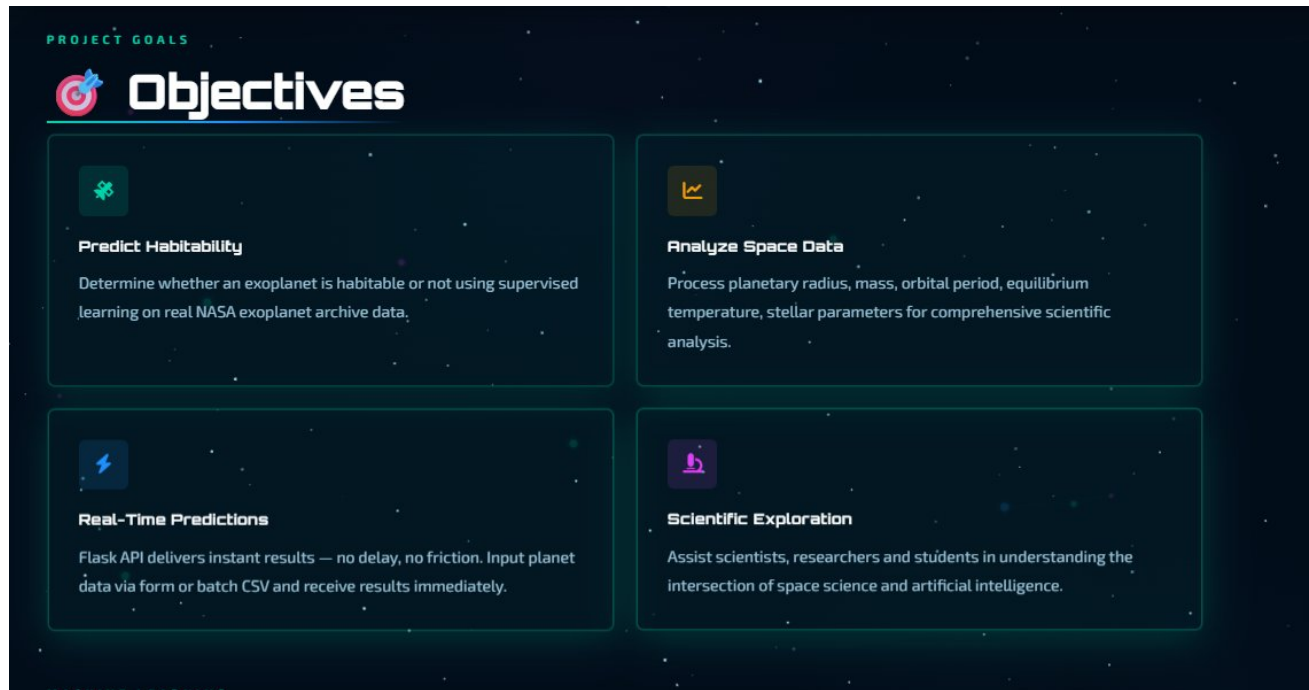


Figure 5: Project Objectives Cards

Screenshot 6 — The Model — Architecture and Performance

The Machine Learning section of the About page showing Model Architecture details (Model Type: Tuned Random Forest, Training: Supervised Learning, Preprocessing: Feature Engineering + Hyperparameter Tuning, Threshold: 0.9, Key Features list) alongside the Model Performance metrics panel displaying all six key metrics with glowing neon numbers.

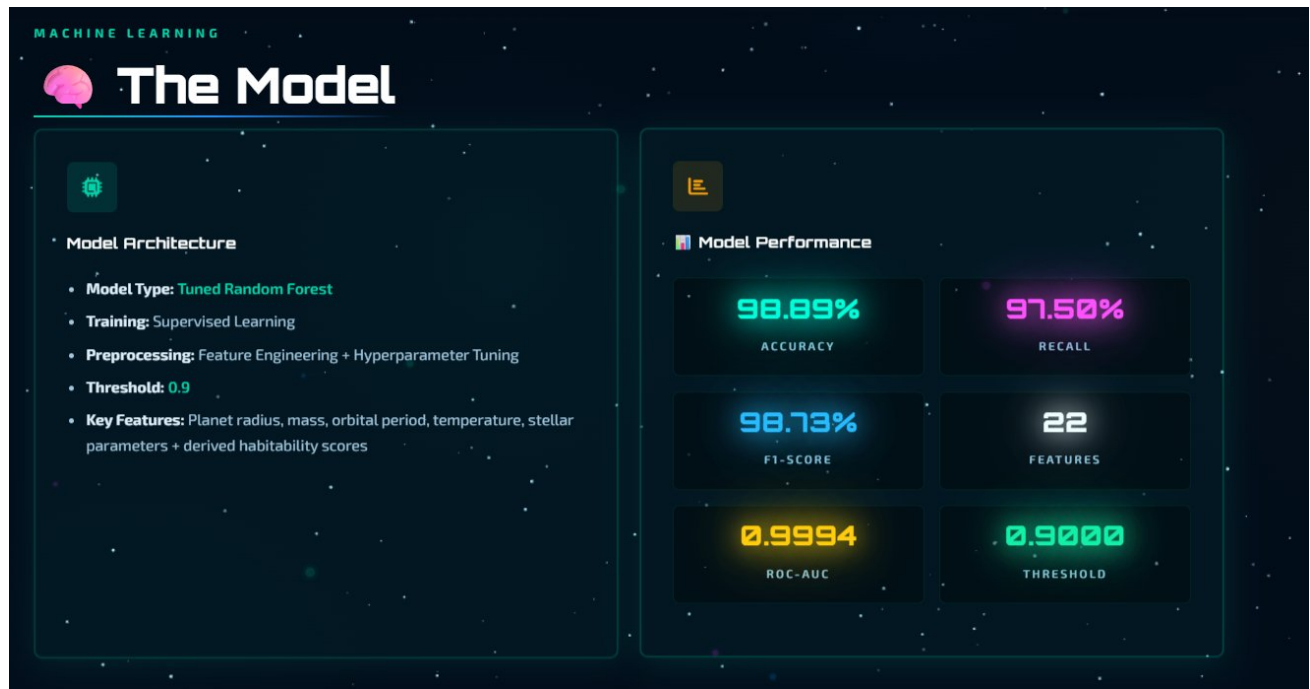


Figure 6: The Model — Architecture and Performance Metrics

Screenshot 7 — Batch Analysis — CSV Upload Tab

The Batch Analysis section showing the CSV upload tab with the drag-and-drop zone, the Run Batch Analysis button, and the sample data download buttons for habitable and non-habitable planet datasets. Also shows the How It Works sidebar explaining the three-step process.

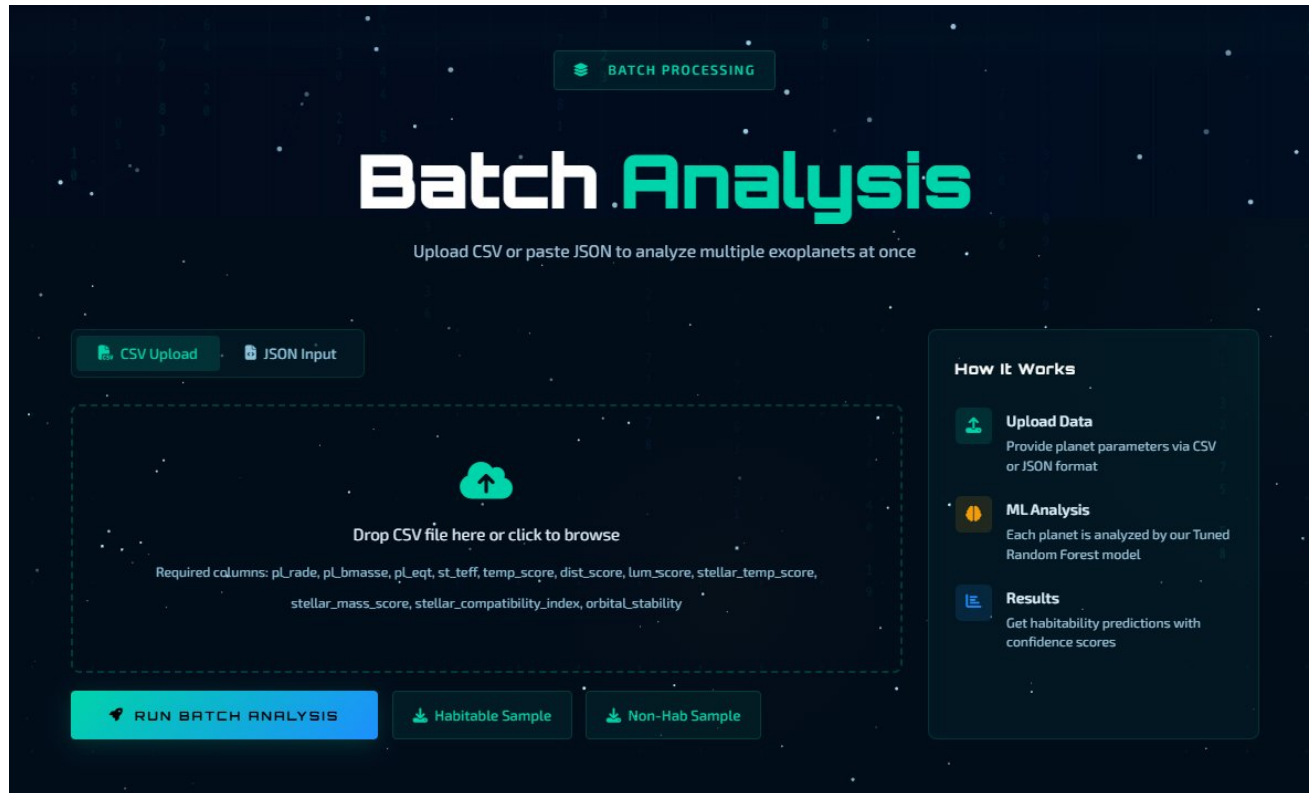


Figure 7: Batch Analysis — CSV Upload Tab

Screenshot 8 — Batch Analysis — JSON Results Table

The batch analysis results table showing the JSON input mode with a sample planet dataset loaded. The summary row displays total analyzed (5), habitable count (5), non-habitable count (0), and analyzed ratio (5/5). The results table lists the top 4 planets with their habitability status and confidence scores.

The interface displays a JSON input field with the following content:

```
{  "planets": [    {      "pl_name": "Kepler-442b-like",      "pl_rad": 1.2,      "pl_bmasse": 1,      "pl_orbper": 365.25,      "pl_orbsmax": 1,    }  ]}
```

Below the input field are three buttons: **RUN BATCH ANALYSIS**, **Habitable Sample**, and **Non-Hab Sample**.

Summary statistics are shown in four boxes:

- TOTAL**: 5
- HABITABLE**: 5
- NON-HABITABLE**: 0
- ANALYZED**: 5/5

Below the summary is a section titled **TOP RESULTS** with a button **SHOWING TOP 4 OF 5**. The table lists the top 4 planets:

#	PLANET NAME	HABITABILITY	CONFIDENCE
#1	Kepler-442b-like	Habitable	92.0%
#2	ExoEarth-Alpha	Habitable	93.0%
#3	GreenWorld-7	Habitable	92.0%
#4	TRAPPIST-1e-like	Habitable	90.0%

On the right side, a **How It Works** section explains the process:

- Upload Data**: Provide planet parameters via CSV or JSON format.
- ML Analysis**: Each planet is analyzed by our Tuned Random Forest model.
- Results**: Get habitability predictions with confidence scores.

Figure 8: Batch Analysis — JSON Input Mode with Results

Screenshot 9 — Exoplanet Ranking Leaderboard

The Ranking section showing the filter controls (number of results set to 22, status filter showing All, minimum score threshold of 0), and the leaderboard table with all columns: Rank (with medals for top 3), Planet Name, Radius, Mass, Orbital Period, Temperature, Star Temperature, Habitability Score, and Status badge. The top planets are shown with 100.0% habitability scores.

RANK	PLANET NAME	RADIUS (R _⊕)	MASS (M _⊕)	PERIOD (D)	TEMP (K)	STAR (K)	SCORE	STATUS
#1	Planet-1	0.70 R _⊕	0.06 M _⊕	4.2 d	0 K	1 K	100.0%	✓ Habitable
#2	Planet-2	0.47 R _⊕	0.03 M _⊕	6.9 d	0 K	1 K	100.0%	✓ Habitable
#3	Planet-3	0.55 R _⊕	0.05 M _⊕	3.2 d	0 K	1 K	100.0%	✓ Habitable
#4	Planet-4	0.68 R _⊕	0.03 M _⊕	3.2 d	1 K	1 K	100.0%	✓ Habitable

Figure 9: Exoplanet Ranking Leaderboard

Screenshot 10 — Planet Prediction Form

The Predict section showing the complete 22-field input form organized into four groups (Planetary Parameters, Stellar Parameters, Spectral Types, and Habitability Scores), the live input preview strip at the bottom of the form, and the sample data buttons and submit button.

The screenshot displays the 'Habitability Planet Prediction' web form against a dark space background with stars. The title 'Habitability Planet Prediction' is at the top, with 'Habitability' in white and 'Planet Prediction' in green. Below the title, a subtitle reads: 'Enter planetary and stellar parameters to predict habitability using our 98.89% accurate model'. The form is divided into sections by green icons and labels. The 'PLANETARY PARAMETERS' section is expanded, showing five input fields: 'Planet Radius (Earth radii)' with a value of 'e.g. 1.2', 'Planet Mass (Earth masses)' with 'e.g. 1.0', 'Orbital Period (days)' with 'e.g. 365.25', 'Semi-major Axis (AU)' with 'e.g. 1.0', and 'Equilibrium Temperature (K)' with 'e.g. 288'. Below this is the 'STELLAR PARAMETERS' section, which includes four input fields: 'Stellar Temperature (K)' with 'e.g. 5772', 'Stellar Radius (Solar radii)' with 'e.g. 1.0', 'Stellar Mass (Solar masses)' with 'e.g. 1.0', and 'Stellar Metallicity' with 'e.g. 0.0'. A fifth field, 'Stellar Luminosity (Solar)', is partially visible at the bottom with a value of 'e.g. 1.0'.

Figure 10: Planet Prediction Form — Planetary Parameters

Screenshot 11 — Prediction Form — Stellar Parameters and Scores

The lower portion of the Predict form showing the Stellar Parameters section, Spectral Types checkboxes (F-type, G-type, K-type, M-type, M-dwarf), Habitability Scores section with seven derived score fields, and the sample data buttons (Habitable, Non-Habitable, Frozen) plus the Auto Fill and Predict Habitability action buttons.

The screenshot displays a dark-themed web interface for a prediction form. It is organized into several sections:

- STELLAR PARAMETERS**: This section contains five input fields with placeholder text "e.g.":
 - Stellar Temperature (K): e.g. 5772
 - Stellar Radius (Solar radii): e.g. 1.0
 - Stellar Mass (Solar masses): e.g. 1.0
 - Stellar Metallicity: e.g. 0.0
 - Stellar Luminosity (Solar): e.g. 1.0
- SPECTRAL TYPES**: This section features five checkboxes labeled "F-type", "G-type", "K-type", "M-type", and "M-dwarf".
- HABITABILITY SCORES (0-1)**: This section contains seven input fields, each with a range of "0.0-1.0":
 - Temperature Score
 - Distance Score
 - Luminosity Score
 - Stellar Temp Score
 - Stellar Mass Score
 - Stellar Compatibility Index
 - Orbital Stability
- Sample Data**: A row of five buttons: "Radius", "Eq. Temp", "Orb. Period", "Star Temp", and "Mass".
- Sample Data**: A row of three buttons: "Habitable" (green), "Non-Habitable" (red), and "Frozen" (blue).
- Actions**: Two buttons at the bottom right: "Auto Fill" (green) and "PREDICT HABITABILITY" (blue).

Figure 11: Prediction Form — Stellar Parameters, Spectral Types & Habitability Scores

Screenshot 12 — Analytics Dashboard — Statistics Overview

The Analytics Dashboard section showing the full hero row (Live Analytics header with Auto-Sync indicator, export buttons: PDF Report, CSV Data, JSON, Refresh, Reset), the four top statistic cards (Total Analyzed: 6, Habitable Candidates: 2, Model Accuracy: 98.89%, Recall Rate: 97.5%), and the four model insight cards (Features: 22, F1-Score: 98.73%, ROC-AUC: 0.9994, Threshold: 0.5).

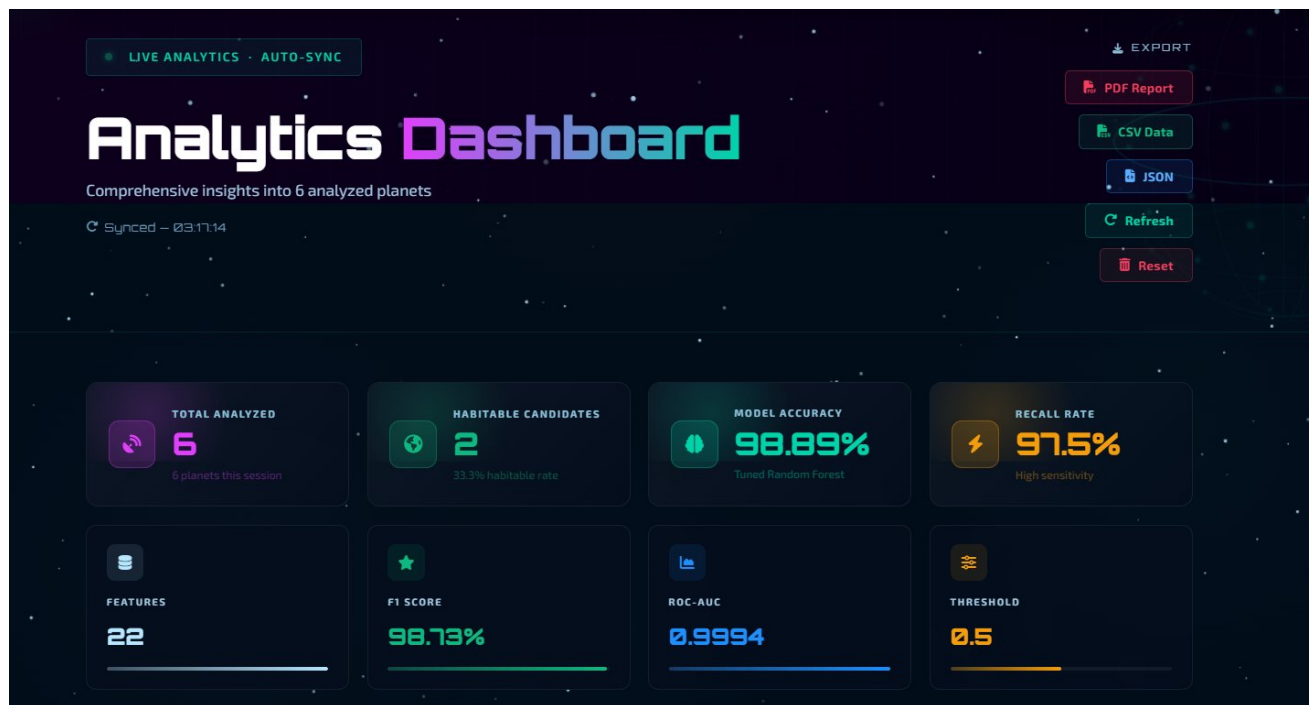


Figure 12: Analytics Dashboard — Statistics Overview

Screenshot 13 — Feature Importance Chart

The Feature Importance horizontal bar chart showing the relative contribution of each feature to the model's habitability predictions. Planet Radius (pl_rade) leads at 43.46%, followed by Planet Mass (pl_bmasse) at 24.45%, with the remaining features contributing smaller but meaningful percentages.



Figure 13: Feature Importance — Key Predictors

Screenshot 14 — Habitability Score Distribution

The Habitability Score Distribution area chart showing the probability distribution of predictions across all analyzed planets. The chart displays concentrations in the 10-20% range (4 planets) and 90-100% range (2 planets), reflecting the bimodal nature of habitability classification.



Figure 14: Habitability Score Distribution Chart

Screenshot 15 — Mass vs Orbital Distance Scatter Plot

The Mass vs Orbital Distance scatter plot where each point represents an analyzed planet positioned by Planet Mass (Earth masses) on the Y-axis and Semi-major Axis (AU) on the X-axis. Habitable planets are shown in green, non-habitable in red, illustrating the relationship between orbital distance and planet mass in habitability classification.

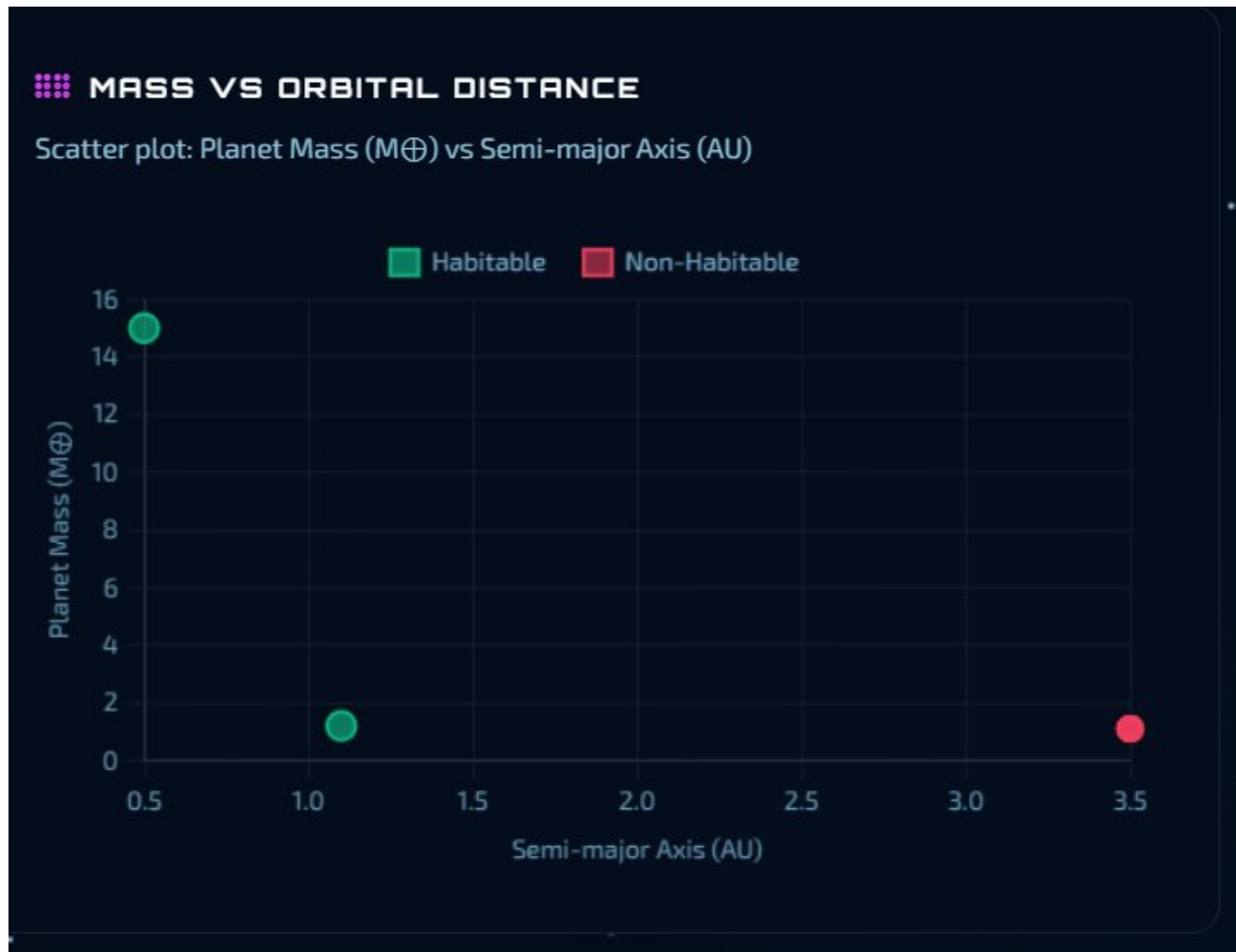


Figure 15: Mass vs Orbital Distance Scatter Plot

Screenshot 16 — Model Performance Metrics

The Model Performance Metrics section with animated progress bars for Accuracy (98.89%), Recall (97.5%), Features (22), F1 Score (98.73%), and ROC-AUC (99.94%). The optimised threshold card shows 0.9000 and the model type card confirms 'Random Forest'.

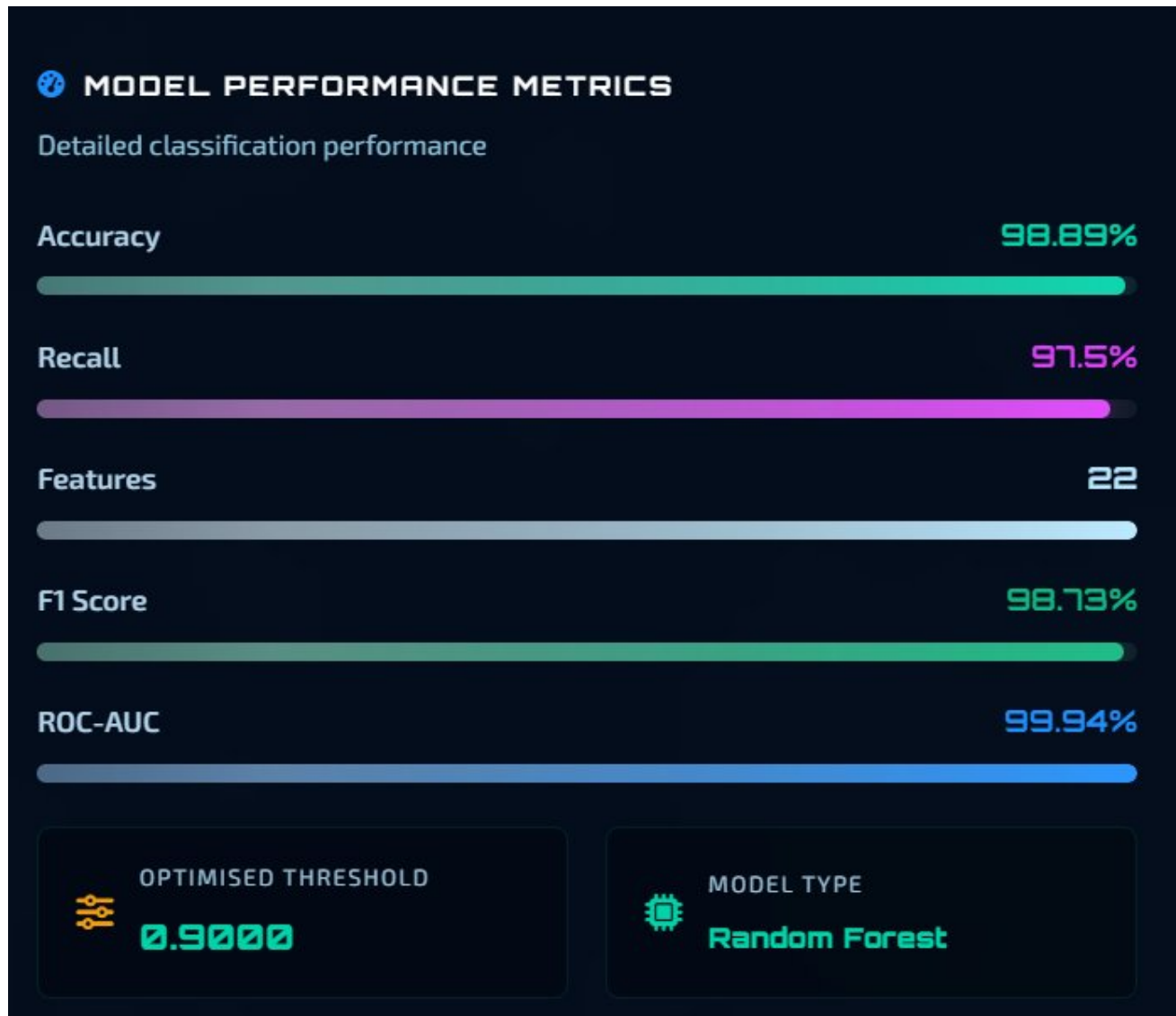


Figure 16: Model Performance Metrics — Detailed Classification Results

6. Summary

Milestone 3 of the ExoHabitAI project successfully delivers a complete, functional end-to-end system connecting a machine learning backend to an interactive web frontend. The backend provides a robust Flask REST API with two primary endpoints — /predict for single-planet habitability inference and /rank for sorted exoplanet leaderboard data — protected by comprehensive input validation and error handling.

The Tuned Random Forest model achieves exceptional performance with 98.89% accuracy, 97.5% recall, 98.73% F1-score, and 0.9994 ROC-AUC. The decision to use a 0.9 probability threshold ensures that every habitable classification made by ExoHabitAI is a high-confidence scientific result.

The frontend delivers a visually immersive experience through its multi-layer canvas animation engine, smooth scroll-triggered effects, and a comprehensive data visualization dashboard powered by Chart.js. Users can analyze single planets or upload CSV files for batch analysis, view ranked leaderboards of habitable candidates, and explore live analytics of their prediction session.

Milestone 3 Deliverable	Status
Flask Backend with /predict endpoint	✓ Complete
Flask Backend with /rank endpoint	✓ Complete
Input Validation for all 22 features	✓ Complete
Tuned Random Forest Model (0.9 threshold)	✓ Complete
6-Section Single-Page Frontend Application	✓ Complete
Single Planet Prediction Form and Results	✓ Complete
Batch CSV and JSON Analysis	✓ Complete
Exoplanet Ranking Leaderboard	✓ Complete
Live Analytics Dashboard with 6 Chart.js Charts	✓ Complete
16 Frontend Screenshots Embedded	✓ Complete
